Rust: Expressions vs Statements

Based on ECM2433 rust_02_crash_course slides and Worksheet 1. Covers the expression/statement distinction, how semicolons affect return values, blocks as expressions, if as an expression, functions with implicit returns, the unit type, and loop returning a value.

What You Need To Know

1. Statements and expressions

Statements perform an action and do not produce a value. **Expressions** evaluate to a value. In Rust, adding a semicolon to the end of an expression turns it into a statement — it discards the value and produces the unit type `()` instead.

```
let x = 5;        // statement: let binding, no value produced
x + 1             // expression: evaluates to 6
x + 1;            // now a statement: value discarded, produces ()
```

2. Blocks are expressions

A block `{ ... }` is itself an expression. Its value is the final expression inside it. If the last line ends with a semicolon, the block evaluates to `()`.

```
let y = {
    let x = 3;
    x + 1          // no semicolon: block evaluates to 4
};                 // y is 4

let z = {
    let x = 3;
    x + 1;         // semicolon: block evaluates to ()
};                 // z is ()
```

3. Functions return their final expression

A function's return value is the value of its final expression. No `return` keyword is needed. Adding a semicolon to the last line changes the return type to `()`, which causes a compile error if the function declares a non-unit return type.

```
fn add(x: i32, y: i32) -> i32 {
    x + y          /* implicit return: no semicolon */
}

fn broken(x: i32) -> i32 {
    x + 10;        /* semicolon: returns () — compile error! */
}
```

This is the exact error shown in Worksheet 1, section 1.3: `fn add_ten(x: i32) -> i32 { x + 10; }` reports "mismatched types: expected i32, found ()".

4. if is an expression

An `if` block evaluates to a value. All branches must return the same type. This lets you write conditional assignments as a single `let` binding.

```
let number = 5;

/* if used as an expression */
let description = if number > 0 {
    "positive"
} else if number < 0 {
    "negative"
} else {
    "zero"
};

/* all branches must return the same type */
let bad = if number > 0 { 1 } else { "oops" }; /* compile error */
```

5. loop can return a value

A `loop` block is also an expression. Use `break value;` to exit the loop and produce a value.

```
let mut counter = 0;
let result = loop {
    counter += 1;
    if counter == 10 {
        break counter * 2;   /* loop evaluates to 20 */
    }
};                           /* result is 20 */
```

6. The unit type ()

`()` is the unit type. Functions with no `->` annotation return `()` implicitly. Writing `-> ()` is valid but rarely needed.

```
fn greet() {                /* implicitly returns () */
    println!("hello");
}

fn explicit_unit() -> () {  /* same thing, explicit */
    println!("hello");
}
```

Practice Questions

Question 1 — this function fails to compile:

```
fn add_ten(x: i32) -> i32 {
    x + 10;
}
```

1. Identify the error and write the corrected function.
2. What type does z have? `let z = { let x = 3; x + 1; };`

Question 3 — state what this prints:

```
let x = {
    let a = 5;
    let b = 3;
    a + b
};
println!("{}", x);
```

3. State what the code above prints.
4. Write `fn larger(a: i32, b: i32) -> i32` that returns the larger value. Use `if` as an expression with no explicit `return` keyword.

Question 5 — this function does not compile:

```
fn classify(score: i32) -> &'static str {
    if score >= 70 {
        "distinction"
    } else if score >= 50 {
        "pass"
    } else {
        0   /* wrong type */
    }
}
```

5. Explain why it does not compile and write a corrected version that returns `"fail"` for scores below 50.

Question 6 — state what this prints:

```
fn double(x: i32) -> i32 { x * 2 }

fn main() {
    let a = double(4);
    let b = {
        let n = double(3);
        n + 1
    };
    println!("{} {}", a, b);
}
```

6. State what the program above prints.
7. What is the return type of a function declared as `fn greet(name: &str)` with no `->` annotation? What would happen if you wrote `let x: i32 = greet("Alice");`?

Question 8 — state the value of result:

```
let mut n = 1;
let result = loop {
    n *= 2;
    if n > 1000 {
        break n;
    }
};
```

8. State the value of `result`.

Question 9 — rewrite this using if as an expression:

```
let label;
if temperature > 100.0 {
    label = "hot";
} else {
    label = "cool";
}
```

9 Rewrite the code above as a single `let` binding.

10 Write `fn sign(n: i32) -> &'static str` that returns `"positive"`, `"negative"`, or `"zero"` using `if/else if/else` as a single expression with no `return` keyword.

Mark Scheme

Attempt all questions before checking.

1 The semicolon on `x + 10;` turns the expression into a statement returning `()`, but the function declares `-> i32`. Fix: remove the semicolon: `fn add_ten(x: i32) -> i32 { x + 10 }`

2 `()` — the semicolon discards the value of `x + 1`.

3 Prints `8` — the block evaluates to `5 + 3 = 8` (no semicolon on last line).

4 `fn larger(a: i32, b: i32) -> i32 { if a > b { a } else { b } }`

5 The final branch returns `0` (integer) while the other branches return `&str` — all branches of an if expression must return the same type. Fix the last branch to return `"fail"`.

6 Prints `8 7`. `double(4)` = 8. Block: `n = double(3) = 6`, then `n + 1 = 7`.

7 Return type is `()`. `let x: i32 = greet("Alice");` would not compile because the function returns `()`, not `i32`.

8 1024 — the loop doubles n each iteration (2, 4, 8, …, 512, 1024) and breaks when n exceeds 1000.

9 `let label = if temperature > 100.0 { "hot" } else { "cool" };`

10 `fn sign(n: i32) -> &'static str { if n > 0 { "positive" } else if n < 0 { "negative" } else { "zero" } }`